

TripLink_Server

Серверная часть приложения TripLink.
Создано системой Doxygen 1.9.8

1 Иерархический список классов	1
1.1 Иерархия классов	1
2 Алфавитный указатель классов	3
2.1 Классы	3
3 Список файлов	5
3.1 Файлы	5
4 Классы	7
4.1 Класс Database	7
4.1.1 Подробное описание	8
4.1.2 Методы	8
4.1.2.1 addUser()	8
4.1.2.2 bookTrip()	9
4.1.2.3 checkUser()	9
4.1.2.4 createTable()	9
4.1.2.5 findTrips()	10
4.1.2.6 getInstance()	10
4.1.2.7 getTrips()	10
4.1.2.8 getUserRating()	10
4.1.2.9 saveRating()	11
4.1.2.10 saveReview()	11
4.1.2.11 saveTrip()	11
4.2 Класс MyTcpServer	12
4.2.1 Подробное описание	13
4.2.2 Конструктор(ы)	13
4.2.2.1 MyTcpServer()	13
4.2.3 Методы	14
4.2.3.1 slotClientDisconnected	14
4.2.3.2 slotNewConnection	14
4.2.3.3 slotServerRead	14
4.3 Класс ServerFunction	14
4.3.1 Подробное описание	15
4.3.2 Методы	15
4.3.2.1 handleAuth()	15
4.3.2.2 handleBookTrip()	15
4.3.2.3 handleCheck()	15
4.3.2.4 handleFindTrip()	16
4.3.2.5 handleRating()	16
4.3.2.6 handleReg()	16
4.3.2.7 handleReview()	16
4.3.2.8 handleStat()	17
4.3.2.9 handleTrip()	17

5 Файлы	19
5.1 Файл server/database.cpp	19
5.1.1 Подробное описание	19
5.2 Файл server/database.h	19
5.2.1 Подробное описание	20
5.3 database.h	20
5.4 Файл server/main.cpp	21
5.4.1 Подробное описание	21
5.4.2 Функции	22
5.4.2.1 main()	22
5.5 Файл server/mytcpserver.cpp	22
5.5.1 Подробное описание	22
5.6 Файл server/mytcpserver.h	22
5.6.1 Подробное описание	23
5.7 mytcpserver.h	23
5.8 Файл server/serverfunction.cpp	24
5.8.1 Подробное описание	24
5.9 Файл server/serverfunction.h	24
5.9.1 Подробное описание	25
5.10 serverfunction.h	25
Предметный указатель	27

Глава 1

Иерархический список классов

1.1 Иерархия классов

Иерархия классов.

QObject	
Database	7
MyTcpServer	12
ServerFunction	14

Глава 2

Алфавитный указатель классов

2.1 Классы

Классы с их кратким описанием.

Database	Класс для управления базой данных, реализующий паттерн Singleton	7
MyTcpServer	Класс, реализующий TCP-сервер для обработки входящих соединений и команд	12
ServerFunction	Класс, содержащий статические методы для обработки команд, получаемых сервером	14

Глава 3

Список файлов

3.1 Файлы

Полный список файлов.

server/ database.cpp	Реализация класса Database для работы с базой данных	19
server/ database.h	Заголовочный файл класса Database для работы с базой данных	19
server/ main.cpp	Главный файл приложения, запускающий TCP сервер	21
server/ mytcpserver.cpp	Реализация класса MyTcpServer , реализующего TCP-сервер	22
server/ mytcpserver.h	Заголовочный файл для класса MyTcpServer , реализующего TCP-сервер	22
server/ serverfunction.cpp	Реализация класса ServerFunction , содержащего обработчики команд сервера . .	24
server/ serverfunction.h	Объявление класса ServerFunction , содержащего обработчики команд сервера . .	24

Глава 4

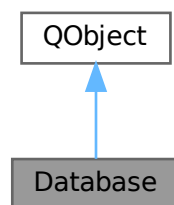
Классы

4.1 Класс Database

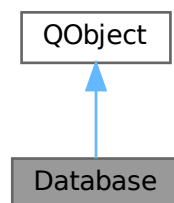
Класс для управления базой данных, реализующий паттерн Singleton.

```
#include <database.h>
```

Граф наследования:Database:



Граф связей класса Database:



Открытые члены

- bool [createTable](#) ()
Создает таблицы в базе данных, если они не существуют.
- bool [addUser](#) (const QStringList &userData)
Добавляет нового пользователя в базу данных.
- bool [checkUser](#) (const QString &login, const QString &password)
Проверяет существование пользователя в базе данных.
- bool [saveTrip](#) (int userId, const QString &from, const QString &to, const QString &time)
Сохраняет поездку в базе данных.
- bool [saveRating](#) (int tripId, int rating)
Сохраняет рейтинг поездки.
- bool [saveReview](#) (int tripId, const QString &review)
Сохраняет отзыв о поездке.
- QVector< QVariant > [getTrips](#) (const QString &login)
Получает список поездок пользователя по его логину.
- double [getUserRating](#) (int userId)
Получает средний рейтинг пользователя.
- QVector< QVariantMap > [findTrips](#) (const QString &from, const QString &to)
Ищет поездки по заданным местам отправления и назначения.
- bool [bookTrip](#) (int tripId, const QString &passengerLogin)
Бронирует поездку для пассажира.

Открытые статические члены

- static [Database](#) & [getInstance](#) ()
Получает единственный экземпляр класса [Database](#).

4.1.1 Подробное описание

Класс для управления базой данных, реализующий паттерн Singleton.

4.1.2 Методы

4.1.2.1 addUser()

```
bool Database::addUser (
    const QStringList & userData )
```

Добавляет нового пользователя в базу данных.

Аргументы

userData	Список данных пользователя (логин, пароль, email).
----------	--

Возвращает

true, если пользователь успешно добавлен, false в случае ошибки.

4.1.2.2 bookTrip()

```
bool Database::bookTrip (
    int tripId,
    const QString & passengerLogin )
```

Бронирует поездку для пассажира.

Аргументы

tripId	ID поездки.
passengerLogin	Логин пассажира.

Возвращает

true, если бронирование успешно, false в случае ошибки.

4.1.2.3 checkUser()

```
bool Database::checkUser (
    const QString & login,
    const QString & password )
```

Проверяет существование пользователя в базе данных.

Аргументы

login	Логин пользователя.
password	Пароль пользователя.

Возвращает

true, если пользователь найден, false в противном случае.

4.1.2.4 createTable()

```
bool Database::createTable ( )
```

Создает таблицы в базе данных, если они не существуют.

Возвращает

true, если таблицы успешно созданы, false в случае ошибки.

4.1.2.5 findTrips()

```
QVector< QVariantMap > Database::findTrips (
    const QString & from,
    const QString & to )
```

Ищет поездки по заданным местам отправления и назначения.

Аргументы

from	Место отправления.
to	Место назначения.

Возвращает

Вектор найденных поездок в виде QVariantMap.

4.1.2.6 getInstance()

```
Database & Database::getInstance ( ) [static]
```

Получает единственный экземпляр класса [Database](#).

Возвращает

Ссылка на экземпляр класса [Database](#).

4.1.2.7 getTrips()

```
QVector< QVariant > Database::getTrips (
    const QString & login )
```

Получает список поездок пользователя по его логину.

Аргументы

login	Логин пользователя.
-------	---------------------

Возвращает

Вектор поездок, представленный в виде QVariant.

4.1.2.8 getUserRating()

```
double Database::getUserRating (
    int userId )
```

Получает средний рейтинг пользователя.

Аргументы

userId	ID пользователя.
--------	------------------

Возвращает

Средний рейтинг пользователя или 0.0 в случае ошибки.

4.1.2.9 saveRating()

```
bool Database::saveRating (
    int tripId,
    int rating )
```

Сохраняет рейтинг поездки.

Аргументы

tripId	ID поездки.
rating	Оценка поездки (число).

Возвращает

true, если рейтинг успешно сохранен, false в случае ошибки.

4.1.2.10 saveReview()

```
bool Database::saveReview (
    int tripId,
    const QString & review )
```

Сохраняет отзыв о поездке.

Аргументы

tripId	ID поездки.
review	Текст отзыва.

Возвращает

true, если отзыв успешно сохранен, false в случае ошибки.

4.1.2.11 saveTrip()

```
bool Database::saveTrip (
    int userId,
```

```
const QString & from,
const QString & to,
const QString & time )
```

Сохраняет поездку в базе данных.

Аргументы

userId	ID пользователя (водителя).
from	Место отправления.
to	Место назначения.
time	Время отправления.

Возвращает

true, если поездка успешно сохранена, false в случае ошибки.

Объявления и описания членов классов находятся в файлах:

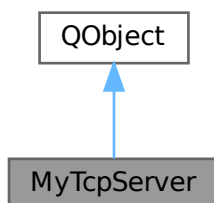
- [server/database.h](#)
- [server/database.cpp](#)

4.2 Класс MyTcpServer

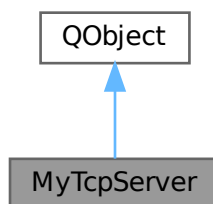
Класс, реализующий TCP-сервер для обработки входящих соединений и команд.

```
#include <mytcpserver.h>
```

Граф наследования:MyTcpServer:



Граф связей класса MyTcpServer:



Открытые слоты

- `void slotNewConnection ()`
Слот, вызываемый при новом входящем соединении.
- `void slotServerRead ()`
Слот для обработки входящих сообщений от клиента.
- `void slotClientDisconnected ()`
Слот, вызываемый при отключении клиента.

Открытые члены

- `MyTcpServer (QObject *parent=nullptr)`
Конструктор класса `MyTcpServer`.

4.2.1 Подробное описание

Класс, реализующий TCP-сервер для обработки входящих соединений и команд.

4.2.2 Конструктор(ы)

4.2.2.1 MyTcpServer()

```
MyTcpServer::MyTcpServer (
    QObject * parent = nullptr ) [explicit]
```

Конструктор класса `MyTcpServer`.

Аргументы

parent	Указатель на родительский объект (по умолчанию nullptr).
--------	--

Инициализирует сервер и начинает прослушивание порта 6000.

4.2.3 Методы

4.2.3.1 slotClientDisconnected

```
void MyTcpServer::slotClientDisconnected ( ) [slot]
```

Слот, вызываемый при отключении клиента.

Закрывает соединение с клиентом.

4.2.3.2 slotNewConnection

```
void MyTcpServer::slotNewConnection ( ) [slot]
```

Слот, вызываемый при новом входящем соединении.

Принимает соединение, отправляет приветственное сообщение клиенту и подключает обработчики событий для чтения данных и отключения клиента.

4.2.3.3 slotServerRead

```
void MyTcpServer::slotServerRead ( ) [slot]
```

Слот для обработки входящих сообщений от клиента.

Обрабатывает команды от клиента, передавая их соответствующим функциям.

Объявления и описания членов классов находятся в файлах:

- [server/mytcpserver.h](#)
- [server/mytcpserver.cpp](#)

4.3 Класс ServerFunction

Класс, содержащий статические методы для обработки команд, получаемых сервером.

```
#include <serverfunction.h>
```

Открытые статические члены

- static void [handleAuth](#) (QTcpSocket *socket, const QString &message)
Обрабатывает команду аутентификации пользователя.
- static void [handleReg](#) (QTcpSocket *socket, const QString &message)
Обрабатывает команду регистрации нового пользователя.
- static void [handleStat](#) (QTcpSocket *socket, const QString &message)
Обрабатывает команду получения статистики поездок пользователя.
- static void [handleCheck](#) (QTcpSocket *socket, const QString &message)
Обрабатывает команду проверки задания.
- static void [handleTrip](#) (QTcpSocket *socket, const QString &message)
Обрабатывает команду создания поездки.
- static void [handleRating](#) (QTcpSocket *socket, const QString &message)
Обрабатывает команду сохранения рейтинга поездки.
- static void [handleReview](#) (QTcpSocket *socket, const QString &message)
Обрабатывает команду сохранения отзыва о поездке.
- static void [handleFindTrip](#) (QTcpSocket *socket, const QString &message)
Обрабатывает команду поиска поездок по заданным направлениям.
- static void [handleBookTrip](#) (QTcpSocket *socket, const QString &message)
Обрабатывает команду бронирования поездки.

4.3.1 Подробное описание

Класс, содержащий статические методы для обработки команд, получаемых сервером.

4.3.2 Методы

4.3.2.1 handleAuth()

```
void ServerFunction::handleAuth (
    QTcpSocket * socket,
    const QString & message ) [static]
```

Обрабатывает команду аутентификации пользователя.

Аргументы

socket	Указатель на сокет клиента.
message	Сообщение с данными для аутентификации.

4.3.2.2 handleBookTrip()

```
void ServerFunction::handleBookTrip (
    QTcpSocket * socket,
    const QString & message ) [static]
```

Обрабатывает команду бронирования поездки.

Аргументы

socket	Указатель на сокет клиента.
message	Сообщение с ID поездки и логином пассажира.

4.3.2.3 handleCheck()

```
void ServerFunction::handleCheck (
    QTcpSocket * socket,
    const QString & message ) [static]
```

Обрабатывает команду проверки задания.

Аргументы

socket	Указатель на сокет клиента.
message	Сообщение с параметрами проверки.

4.3.2.4 handleFindTrip()

```
void ServerFunction::handleFindTrip (
    QTcpSocket * socket,
    const QString & message ) [static]
```

Обрабатывает команду поиска поездок по заданным направлениям.

Аргументы

socket	Указатель на сокет клиента.
message	Сообщение с пунктами отправления и назначения.

4.3.2.5 handleRating()

```
void ServerFunction::handleRating (
    QTcpSocket * socket,
    const QString & message ) [static]
```

Обрабатывает команду сохранения рейтинга поездки.

Аргументы

socket	Указатель на сокет клиента.
message	Сообщение с ID поездки и рейтингом.

4.3.2.6 handleReg()

```
void ServerFunction::handleReg (
    QTcpSocket * socket,
    const QString & message ) [static]
```

Обрабатывает команду регистрации нового пользователя.

Аргументы

socket	Указатель на сокет клиента.
message	Сообщение с данными для регистрации.

4.3.2.7 handleReview()

```
void ServerFunction::handleReview (
    QTcpSocket * socket,
    const QString & message ) [static]
```

Обрабатывает команду сохранения отзыва о поездке.

Аргументы

socket	Указатель на сокет клиента.
message	Сообщение с ID поездки и отзывом.

4.3.2.8 handleStat()

```
void ServerFunction::handleStat (
    QTcpSocket * socket,
    const QString & message ) [static]
```

Обрабатывает команду получения статистики поездок пользователя.

Аргументы

socket	Указатель на сокет клиента.
message	Сообщение с логином пользователя.

4.3.2.9 handleTrip()

```
void ServerFunction::handleTrip (
    QTcpSocket * socket,
    const QString & message ) [static]
```

Обрабатывает команду создания поездки.

Аргументы

socket	Указатель на сокет клиента.
message	Сообщение с данными о поездке.

Объявления и описания членов классов находятся в файлах:

- [server/serverfunction.h](#)
- [server/serverfunction.cpp](#)

Глава 5

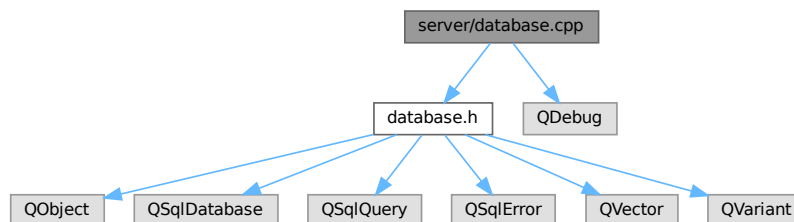
Файлы

5.1 Файл server/database.cpp

Реализация класса `Database` для работы с базой данных.

```
#include "database.h"  
#include <QDebug>
```

Граф включаемых заголовочных файлов для database.cpp:



5.1.1 Подробное описание

Реализация класса `Database` для работы с базой данных.

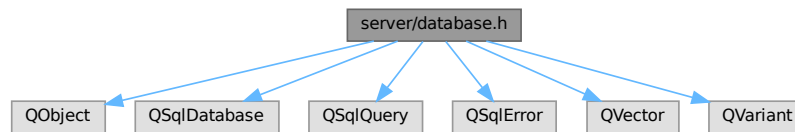
5.2 Файл server/database.h

Заголовочный файл класса `Database` для работы с базой данных.

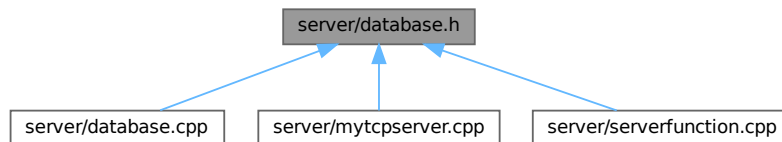
```
#include <QObject>  
#include <QSqlDatabase>  
#include <QSqlQuery>  
#include <QSqlError>  
#include <QVector>
```

```
#include <QVariant>
```

Граф включаемых заголовочных файлов для database.h:



Граф файлов, в которые включается этот файл:



Классы

- class [Database](#)

Класс для управления базой данных, реализующий паттерн Singleton.

5.2.1 Подробное описание

Заголовочный файл класса [Database](#) для работы с базой данных.

5.3 database.h

[См. документацию.](#)

```

00001
00006 #ifndef DATABASE_H
00007 #define DATABASE_H
00008
00009 #include <QObject>
00010 #include <QSqlDatabase>
00011 #include <QSqlQuery>
00012 #include <QSqlError>
00013 #include <QVector>
00014 #include <QVariant>
00015
00020 class Database : public QObject
00021 {
00022     Q_OBJECT
00023
00024 public:
00029     static Database& getInstance();
00030
00035     bool createTable();
00036
00042     bool addUser(const QStringList& userData);
  
```

```

00043
00050     bool checkUser(const QString& login, const QString& password);
00051
00060     bool saveTrip(int userId, const QString& from, const QString& to, const QString& time);
00061
00068     bool saveRating(int tripId, int rating);
00069
00076     bool saveReview(int tripId, const QString& review);
00077
00083     QVector<QVariant> getTrips(const QString& login);
00084
00090     double getUserRating(int userId);
00091
00098     QVector<QVariantMap> findTrips(const QString& from, const QString& to);
00099
00106     bool bookTrip(int tripId, const QString& passengerLogin);
00107
00108 private:
00113     Database(QObject* parent = nullptr);
00114
00118     Database(const Database&) = delete;
00119
00123     Database& operator=(const Database&) = delete;
00124
00125     QSqlDatabase db;
00126 };
00127
00128 #endif // DATABASE_H
00129

```

5.4 Файл server/main.cpp

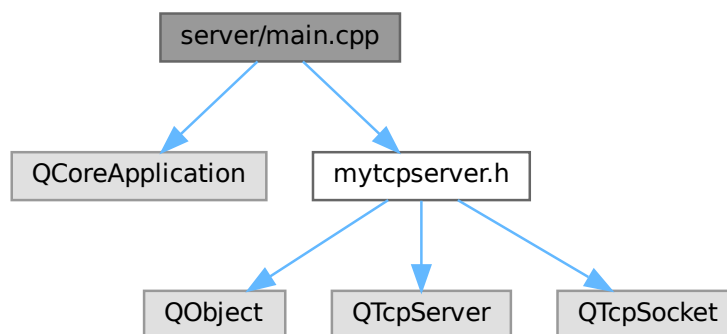
Главный файл приложения, запускающий TCP сервер.

```

#include <QCoreApplication>
#include "mytcpserver.h"

```

Граф включаемых заголовочных файлов для main.cpp:



Функции

- int `main` (int argc, char *argv[])

5.4.1 Подробное описание

Главный файл приложения, запускающий TCP сервер.

5.4.2 Функции

5.4.2.1 main()

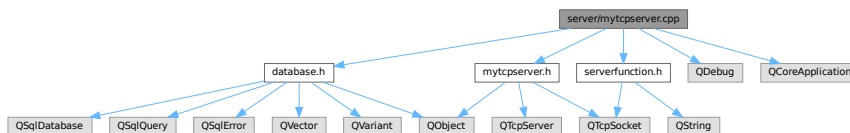
```
int main (
    int argc,
    char * argv[] )
```

5.5 Файл server/mytcpserver.cpp

Реализация класса [MyTcpServer](#), реализующего TCP-сервер.

```
#include "mytcpserver.h"
#include "serverfunction.h"
#include "database.h"
#include <QDebug>
#include <QCoreApplication>
```

Граф включаемых заголовочных файлов для mytcpserver.cpp:



5.5.1 Подробное описание

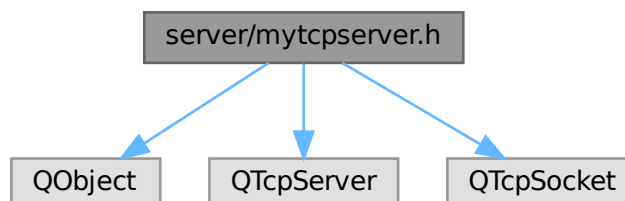
Реализация класса [MyTcpServer](#), реализующего TCP-сервер.

5.6 Файл server/mytcpserver.h

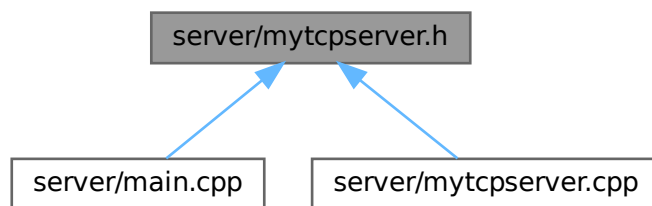
Заголовочный файл для класса [MyTcpServer](#), реализующего TCP-сервер.

```
#include <QObject>
#include <QTcpServer>
#include <QTcpSocket>
```

Граф включаемых заголовочных файлов для mytcpserver.h:



Граф файлов, в которые включается этот файл:



Классы

- class [MyTcpServer](#)
Класс, реализующий TCP-сервер для обработки входящих соединений и команд.

5.6.1 Подробное описание

Заголовочный файл для класса [MyTcpServer](#), реализующего TCP-сервер.

5.7 mytcpserver.h

[См. документацию.](#)

```

00001
00006 #ifndef MYTCPSERVER_H
00007 #define MYTCPSERVER_H
00008
00009 #include <QObject>
00010 #include <QTcpServer>
00011 #include <QTcpSocket>
00012
00017 class MyTcpServer : public QObject
00018 {
00019     Q_OBJECT
00020 public:
00027     explicit MyTcpServer(QObject *parent = nullptr);
00028
00029 public slots:
00036     void slotNewConnection();
00037
00043     void slotServerRead();
00044
00050     void slotClientDisconnected();
00051
00052 private:
00053     QTcpServer *mTcpServer;
00054     QTcpSocket *mTcpSocket;
00055 };
00056
00057 #endif // MYTCPSERVER_H
00058

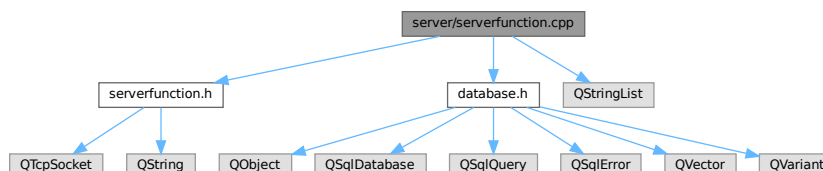
```

5.8 Файл server/serverfunction.cpp

Реализация класса [ServerFunction](#), содержащего обработчики команд сервера.

```
#include "serverfunction.h"
#include "database.h"
#include <QStringList>
```

Граф включаемых заголовочных файлов для serverfunction.cpp:



5.8.1 Подробное описание

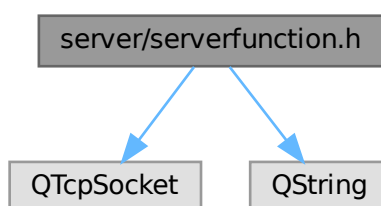
Реализация класса [ServerFunction](#), содержащего обработчики команд сервера.

5.9 Файл server/serverfunction.h

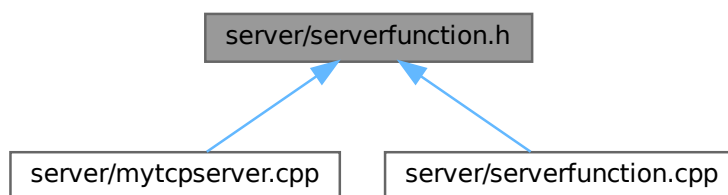
Объявление класса [ServerFunction](#), содержащего обработчики команд сервера.

```
#include <QTcpSocket>
#include <QString>
```

Граф включаемых заголовочных файлов для serverfunction.h:



Граф файлов, в которые включается этот файл:



Классы

- class [ServerFunction](#)

Класс, содержащий статические методы для обработки команд, получаемых сервером.

5.9.1 Подробное описание

Объявление класса [ServerFunction](#), содержащего обработчики команд сервера.

5.10 serverfunction.h

[См. документацию.](#)

```

00001
00006 #ifndef SERVERFUNCTION_H
00007 #define SERVERFUNCTION_H
00008
00009 #include <QTcpSocket>
00010 #include <QString>
00011
00016 class ServerFunction
00017 {
00018 public:
00024     static void handleAuth(QTcpSocket* socket, const QString& message);
00025
00031     static void handleReg(QTcpSocket* socket, const QString& message);
00032
00038     static void handleStat(QTcpSocket* socket, const QString& message);
00039
00045     static void handleCheck(QTcpSocket* socket, const QString& message);
00046
00052     static void handleTrip(QTcpSocket* socket, const QString& message);
00053
00059     static void handleRating(QTcpSocket* socket, const QString& message);
00060
00066     static void handleReview(QTcpSocket* socket, const QString& message);
00067
00073     static void handleFindTrip(QTcpSocket* socket, const QString& message);
00074
00080     static void handleBookTrip(QTcpSocket* socket, const QString& message);
00081 };
00082
00083 #endif // SERVERFUNCTION_H
00084
  
```


Предметный указатель

- addUser
 - Database, [8](#)
- bookTrip
 - Database, [9](#)
- checkUser
 - Database, [9](#)
- createTable
 - Database, [9](#)
- Database, [7](#)
 - addUser, [8](#)
 - bookTrip, [9](#)
 - checkUser, [9](#)
 - createTable, [9](#)
 - findTrips, [9](#)
 - getInstance, [10](#)
 - getTrips, [10](#)
 - getUserRating, [10](#)
 - saveRating, [11](#)
 - saveReview, [11](#)
 - saveTrip, [11](#)
- findTrips
 - Database, [9](#)
- getInstance
 - Database, [10](#)
- getTrips
 - Database, [10](#)
- getUserRating
 - Database, [10](#)
- handleAuth
 - ServerFunction, [15](#)
- handleBookTrip
 - ServerFunction, [15](#)
- handleCheck
 - ServerFunction, [15](#)
- handleFindTrip
 - ServerFunction, [15](#)
- handleRating
 - ServerFunction, [16](#)
- handleReg
 - ServerFunction, [16](#)
- handleReview
 - ServerFunction, [16](#)
- handleStat
 - ServerFunction, [17](#)
- handleTrip
 - ServerFunction, [17](#)
- main
 - main.cpp, [22](#)
- main.cpp
 - main, [22](#)
- MyTcpServer, [12](#)
 - MyTcpServer, [13](#)
 - slotClientDisconnected, [14](#)
 - slotNewConnection, [14](#)
 - slotServerRead, [14](#)
- saveRating
 - Database, [11](#)
- saveReview
 - Database, [11](#)
- saveTrip
 - Database, [11](#)
- server/database.cpp, [19](#)
- server/database.h, [19](#), [20](#)
- server/main.cpp, [21](#)
- server/mytcpserver.cpp, [22](#)
- server/mytcpserver.h, [22](#), [23](#)
- server/serverfunction.cpp, [24](#)
- server/serverfunction.h, [24](#), [25](#)
- ServerFunction, [14](#)
 - handleAuth, [15](#)
 - handleBookTrip, [15](#)
 - handleCheck, [15](#)
 - handleFindTrip, [15](#)
 - handleRating, [16](#)
 - handleReg, [16](#)
 - handleReview, [16](#)
 - handleStat, [17](#)
 - handleTrip, [17](#)
- slotClientDisconnected
 - MyTcpServer, [14](#)
- slotNewConnection
 - MyTcpServer, [14](#)
- slotServerRead
 - MyTcpServer, [14](#)